PROOF OF CONCEPT · FULL-CODE PATH

AuroraStay Hotels Policy Query Assistant

A Retrieval-Augmented Generation system that answers staff questions from the official policy documents - with sources

Aryan Patel · October 2026

The business problem

- **18 properties**, 3-4 added per year - property-specific rules keep growing
- **~1,200 guest queries a week** on rooms, check-in/out, cancellation, refunds, breakfast, parking, pets, charges
- **40 % are complex**: the answer is spread over several documents
- Manual lookup takes **8-12 minutes** and is wrong or incomplete **15-20 %** of the time
- **45 % annual staff turnover** - institutional knowledge keeps leaving

Objective of this stage

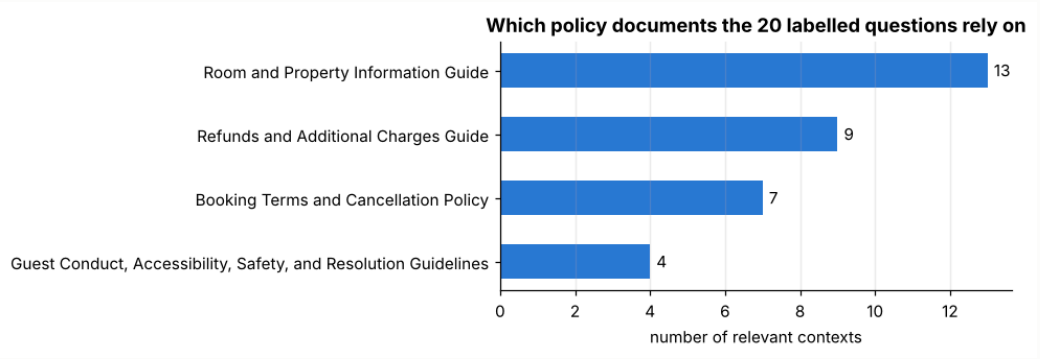
An internal tool where staff type a question and get an answer **grounded only in the policy documents**, with the supporting excerpts shown.

Success = reliability: can a meaningful share of queries be answered without manual lookup or escalation? Speed is not the goal yet.

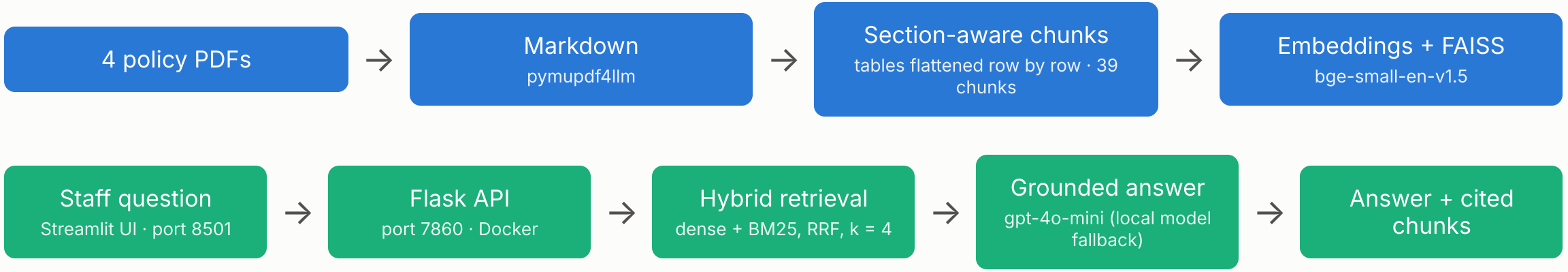
The data

Policy	Code	Pages	Covers
Booking Terms & Cancellation	AS-01	5	rate plans, deadlines, no-shows, deposits, property quick reference
Room & Property Information	AS-02	5	room types, arrival/departure, amenities, parking, pets, service animals, property matrix
Refunds & Additional Charges	AS-03	4	refund scenarios, itemised charges, incidental holds, dispute workflow
Guest Conduct, Accessibility & Resolution	AS-04	5	conduct rules, ADA, safety, complaints, escalation matrix

- Same template in all four; **facts live in 20+ tables**
- Six-property matrix repeated across documents
- **20 labelled questions** (10 evaluation + 10 test); over half need more than one source



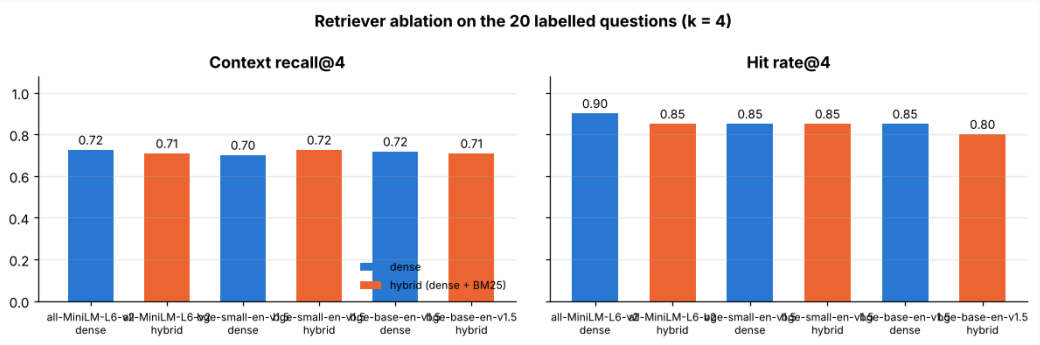
Solution architecture



- Provider-agnostic: an OpenAI-compatible endpoint when a key is configured (as in GitHub Codespaces), otherwise a local open-weights model on CPU - the same code runs fully on-premise
- Strict prompt contract: only the retrieved excerpts, exact figures, never transfer a fee between properties, say "not covered" instead of guessing

Chunking and the embedding model

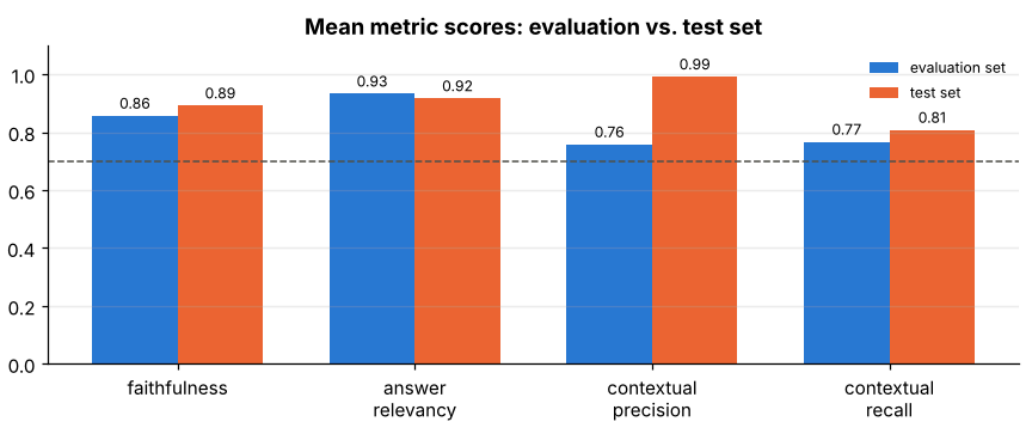
- **One section = one chunk** (numbered headings), with a breadcrumb "Document | Section"
- **Every table row rewritten as a labelled sentence**: "ASH-MIA Brickell: Breakfast: $24 per person" - the model had mis-read a neighbouring row of the raw table
- Long sections split on paragraphs with overlap → **39 chunks**, ~1,000 characters each
- Three CPU-friendly embedding models benchmarked on the 20 labelled questions; **bge-small-en-v1.5** chosen (matches bge-base at a third of the cost; 512-token window)
- **Hybrid retrieval** (dense + BM25) beats dense-only: recall@4 0.70 → 0.72, hit rate 0.85 → 0.85



Evaluation: four DeepEval metrics, judge = gpt-4o-mini

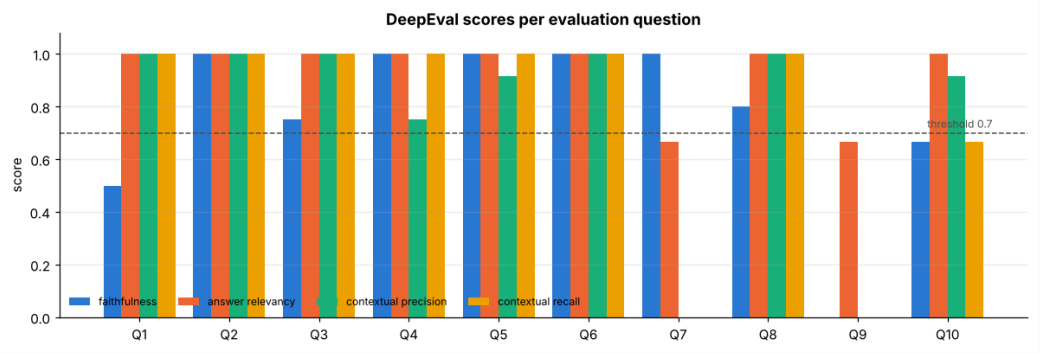
Metric	Evaluation (10)	Test (10)	Pass ≥ 0.7
Faithfulness	0.86	0.89	80%
Answer Relevancy	0.93	0.92	80%
Contextual Precision	0.76	0.99	90%
Contextual Recall	0.77	0.81	65%
Composite	**0.83**	**0.90**	**11/20 all four**

Faithfulness = no contradiction with the retrieved policy text · Answer relevancy = addresses the question · Contextual precision / recall = retrieval ranking / coverage against the expert answer. One or more judge calls timed out during an endpoint stall and are excluded from the means.



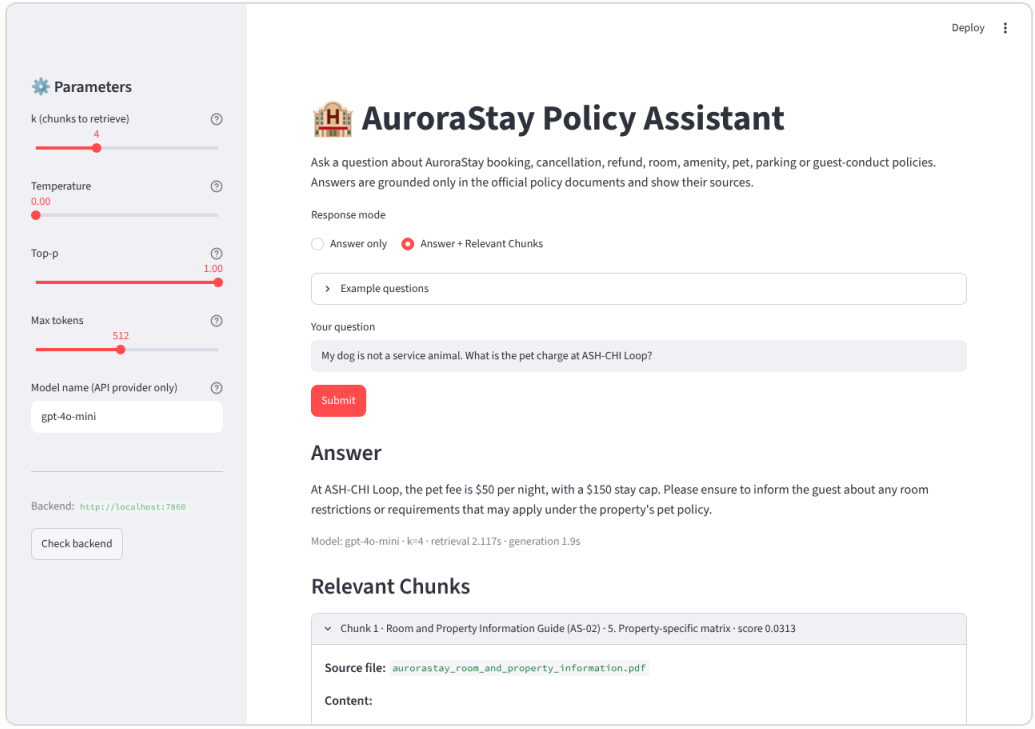
What the results say

- **Grounding works.** Faithfulness 0.88 across 20 questions; the out-of-scope question (Austin North pool) was answered with "the policies do not cover this - check the live profile"
- **Held-out test set tracks the evaluation set** (0.90 vs 0.83) - the design choices generalise
- **Retrieval is where the remaining errors come from**: vocabulary gaps ("loud since midnight" vs "quiet hours") and questions needing two documents
- **Coverage is limited by the documents**: property facts exist for 6 of 18 properties
- **Latency is already workable**: median 2.0 s per answer with the API model; the local CPU fallback needs 30-70 s



Deployment

- **Backend**: serialized FAISS index + `rag_utils.py` + Flask API (`/v1/answer`, `/v1/answer_with_relevant_chunks`, `/v1/relevant_chunks`), pinned requirements, Dockerfile (Gunicorn, port 7860)
- **Frontend**: Streamlit UI with k / temperature / top-p / max-tokens controls, "Answer only" or "Answer + Relevant Chunks", Dockerfile (port 8501)
- Files pushed to GitHub; both containers built and run; online inference through the API URL - in Codespaces the forwarded `https://<name>-7860.app.github.dev`
- Secrets only as environment variables (`OPENAI_API_KEY`, `OPENAI_API_BASE`)



Recommendations

#	Recommendation	Why
1	**Run a 6-8 week internal pilot** at 2-3 properties; staff rate every answer and whether they still had to look something up	Turns the POC score into the business metric: share of queries resolved without manual lookup
2	**Grow the labelled question bank to 100+** from pilot logs; re-run the evaluation weekly with an independent judge	20 questions cannot separate 80 % from 90 % reliability
3	**Ingest live property profiles for all 18 properties** (and every new one at onboarding)	Largest coverage gap, cheapest to close
4	**Add guest-language synonyms to policy sections** and/or a query-rewriting step	Fixes the vocabulary-gap retrieval misses
5	**Keep the API model for the pilot, the local model as on-premise fallback**	Seconds per answer; privacy and outage protection
6	**Keep the guardrails** (temperature 0, "not covered" refusals, sources always shown) and add an audit log	Accuracy and traceability first
7	**Version the knowledge base** by policy effective date; re-index on change; use the tool in new-hire onboarding	Stale chunks become wrong answers; turnover is 45 %

Potential benefit and conclusion

Illustrative benefit

480 complex queries a week × 8-12 minutes = **64-96 staff-hours a week** of manual lookup.

In the POC **11 of 20** labelled questions passed all four quality gates. If the pilot confirms that rate, roughly **264 complex queries a week** (35-53 staff-hours) no longer need a manual search, and the rest come with cited excerpts to verify instead of research.

Conclusion

- End-to-end RAG assistant built, evaluated and containerised: grounded, cited answers; says "not covered" rather than guessing
- Composite quality 0.83 (evaluation) / 0.90 (test) on 20 expert-labelled questions
- Weak spots have concrete, low-cost fixes: synonyms for retrieval, all-property profiles, a larger question bank
- **Next step: the internal pilot**, measuring the share of queries answered reliably without manual lookup